MANUAL DE FreeRTOS PARA ESP32



Instituto Tecnológico de Oaxaca Noviembre 2026

RESPONSABLE:

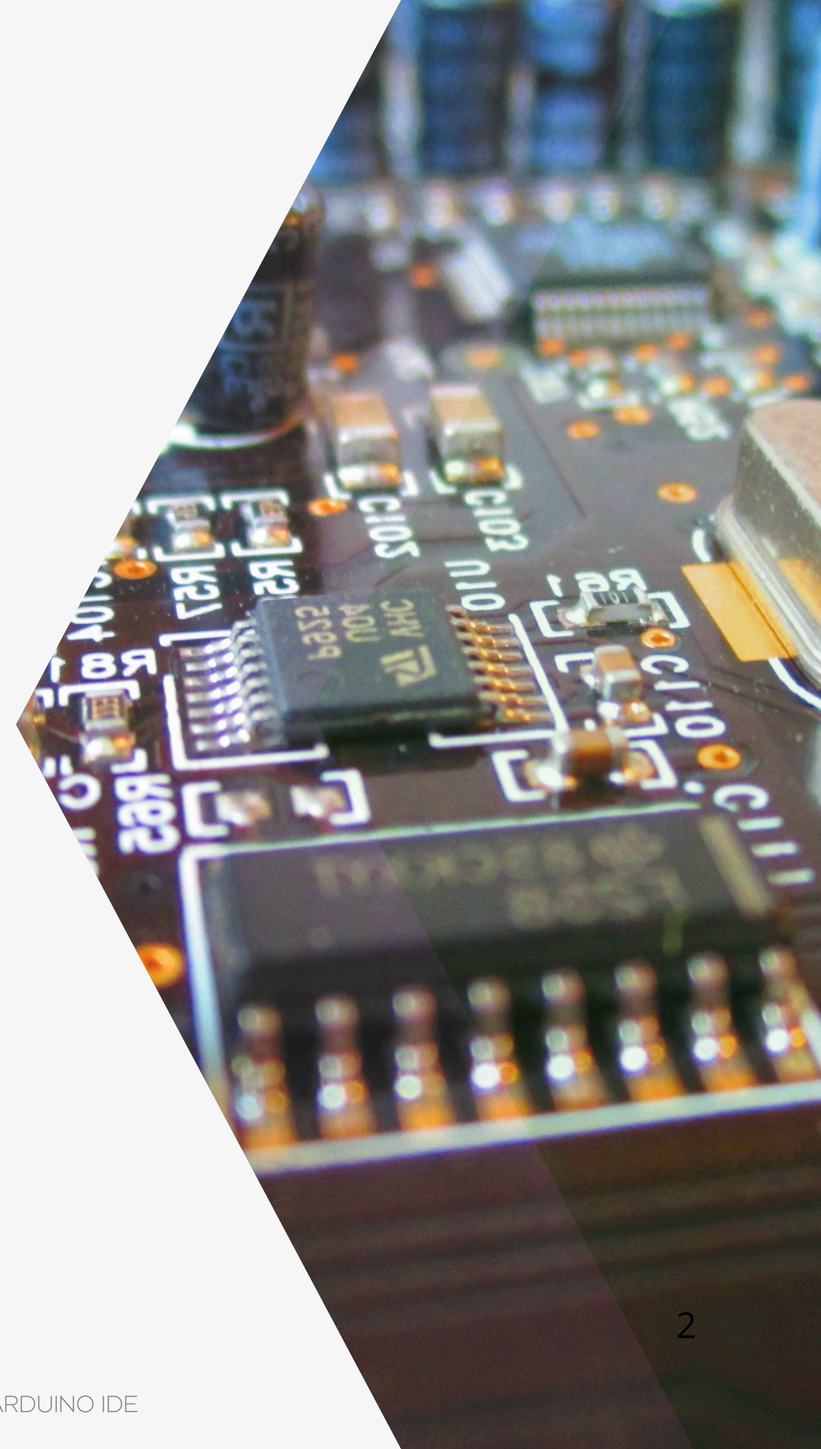
DR. PÉREZ RAMOS ALDO ELEAZAR

COLABORADORES:

ING. LUCERO CRUZ HEBER

ÍNDICE

- Introducción 03
- Instalación y preparación de arduino IDE 10
- Manejo de entradas y salidas digitales 23
- 34
- 47
- 60



INTRODUCCIÓN

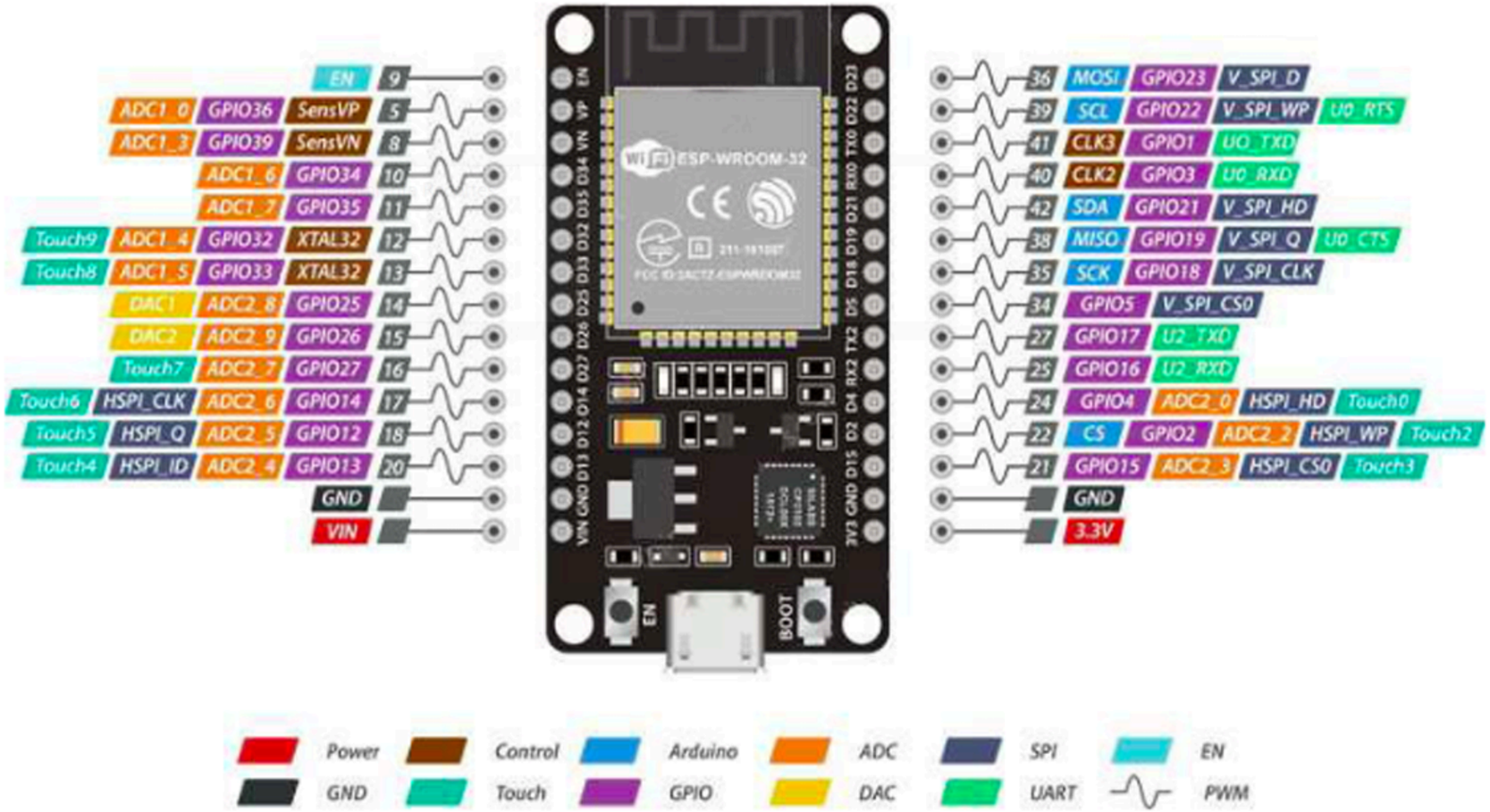
Arduino IDE es un entorno de desarrollo integrado (IDE) que se utiliza ampliamente para programar y desarrollar proyectos con placas Arduino.
Arduino es una plataforma de hardware de código abierto que permite a los usuarios crear proyectos interactivos utilizando microcontroladores programables.
El IDE de Arduino proporciona una interfaz fácil de usar que simplifica la programación y la carga de código en las placas Arduino.



ESP32, es una familia de microcontroladores de bajo costo y bajo consumo de energía que cuenta con conectividad Wi-Fi y Bluetooth integrada.
Estos microcontroladores son muy populares en el ámbito de la electrónica y la programación de dispositivos IoT (Internet de las cosas) debido a su potencia de procesamiento, su versatilidad y su capacidad para conectarse a redes inalámbricas.



PINOUT DE ESP 32 / WROOM 32



Arduino IDE es compatible con una amplia gama de placas Arduino, incluida la ESP32. Esto significa que puedes utilizar Arduino IDE para programar y desarrollar proyectos con el ESP32, aprovechando todas las características y funcionalidades que este microcontrolador ofrece. Con Arduino IDE y ESP32, puedes crear una variedad de proyectos interesantes, como sistemas de monitoreo y control remoto, dispositivos de automatización del hogar, entre otros.
La combinación de la facilidad de uso de Arduino IDE y las capacidades del ESP32 proporciona un entorno de desarrollo poderoso y accesible para crear tus propias aplicaciones y dispositivos electrónicos.

Al utilizar Arduino IDE con ESP32, puedes aprovechar las características y funcionalidades del ESP32 para desarrollar proyectos de IoT y electrónica de manera sencilla y accesible

FRERTOS

Un RTOS es un sistema operativo diseñado para responder a eventos dentro de tiempos conocidos, algo fundamental cuando programamos sistemas embebidos que no pueden quedarse “pensando” indefinidamente.

Vamos a hablar de FreeRTOS y de sistemas operativos en tiempo real aprenderemos su estructura, su funcionamiento y como aplicarlos en nuestros codigos de programación.



Si vienes del mundo de Arduino, es muy probable que estés acostumbrado a programar dentro de setup() y loop(). Es la forma estándar, sencilla y funciona de maravilla para parpadear un LED o leer un sensor.

Pero, a medida que nuestros proyectos crecen, empezamos a encontrarnos con problemas. Queremos leer un sensor cada 100ms, controlar un motor paso a paso, atender peticiones WiFi y actualizar una pantalla, todo “a la vez”.
En ese punto la programación secuencial clásica empieza a hacer aguas y un RTOS se convierte en una herramienta muy útil.



En este curso usaremos principalmente ESP32, ya que utiliza FreeRTOS de forma nativa en su SDK, aunque los conceptos son aplicables a cualquier microcontrolador compatible.

El paradigma del “Super Loop”

Antes de entender la solución, analicemos el problema. La arquitectura clásica de un firmware simple es el Bucle Infinito o “Super Loop”.

Esto es **programación secuencial**. El procesador ejecuta la primera instrucción, luego la segunda, y así sucesivamente.

```
void setup() {
    // Inicialización
}

void loop() {
    leerSensores();
    procesarDatos();
    actualizarPantalla();
    moverMotores();
}
```

El problema del bloqueo

El problema surge cuando una de esas funciones tarda mucho tiempo. Por ejemplo, imagina que usamos la infame función delay():

```
void loop() {
    digitalWrite(LED, HIGH);
    delay(1000); // <--- El procesador se queda AQUÍ bloqueado
    digitalWrite(LED, LOW);
    delay(1000);

    leerBoton(); // <--- Si pulsas el botón durante el delay, el sistema NO se entera
}
```

Mientras el procesador está contando milisegundos en el delay(), no puede hacer nada más. Si llega un dato por el puerto serie, si el usuario pulsa un botón o si hay una emergencia, el microcontrolador está “sordo”.

Solucion comun

Podemos mejorar esto usando millis() y máquinas de estados, pero el código se vuelve complejo, difícil de leer y de mantener a medida que añadimos funcionalidades.

¿Qué es un RTOS?

Un RTOS (Real-Time Operating System) es un sistema operativo diseñado para gestionar los recursos del hardware (CPU y memoria) con tiempos de respuesta predecibles y acotables.

A diferencia de sistemas operativos de propósito general (como Windows o Linux), donde el objetivo es el rendimiento global (throughput), en un RTOS el objetivo principal es el determinismo y la latencia.

Esto nos permite diseñar el sistema para que, si ocurre un evento crítico, reaccione dentro de un plazo conocido. Pero ojo: usar un RTOS no garantiza por sí solo que cumplamos ese plazo. También necesitamos asignar bien las prioridades, evitar bloqueos largos y analizar el peor caso.

¿Por qué es útil FreeRTOS con el ESP32?



FreeRTOS resulta útil con el ESP32 porque permite la multitarea, lo que posibilita que varias tareas, como la lectura de sensores, la conexión Wi-Fi y las actualizaciones de la pantalla, se ejecuten sin bloquearse entre sí. También te permite aprovechar el procesador de doble núcleo del ESP32, ya que puedes asignar tareas a núcleos específicos para obtener un mejor rendimiento.

Además, gracias a la programación basada en prioridades, las tareas críticas en cuanto al tiempo pueden ejecutarse de inmediato, lo que resulta ideal para aplicaciones en tiempo real cuando se necesita reaccionar rápidamente a eventos externos detectados por los GPIO del ESP32.



Conceptos básicos de FreeRTOS

Antes de adentrarnos, repasemos algunos conceptos básicos relacionados con FreeRTOS:

- **Tareas** : Las tareas son funciones independientes que se ejecutan simultáneamente, cada una con su propia pila (uso de memoria asignado) y prioridad. Las tareas pueden estar en estados como En ejecución, Lista, Bloqueada o Suspendida.

- **Planificador** : el planificador decide qué tareas ejecutar según su prioridad. Se trata de un planificador preventivo, lo que significa que puede interrumpir una tarea de menor prioridad en cualquier momento para ejecutar una de mayor prioridad, garantizando así que las tareas críticas se ejecuten en cuanto estén listas.
-
- **Prioridades** : los números más altos indican mayor prioridad (por ejemplo: 1 = baja, 5 = alta).

Tarea

En FreeRTOS, una tarea (Task) es una unidad independiente de ejecución que realiza una función específica, como controlar un LED o leer un sensor. Cada tarea tiene su propia pila de memoria (stack) y una prioridad que influye en cuándo se ejecuta.

Durante su funcionamiento, una tarea puede estar lista (Ready), en ejecución (Running), bloqueada (Blocked) mientras espera un tiempo o evento, o suspendida (Suspended) hasta que se reanude explícitamente.

Idea clave: las tareas permiten dividir un programa en actividades independientes que FreeRTOS organiza y planifica para compartir el procesador.



Planificador

El planificador (Scheduler) es el componente de FreeRTOS que decide qué tarea utiliza el procesador en cada momento.

En la planificación preventiva (preemptive), puede interrumpir una tarea en ejecución cuando otra de mayor prioridad está lista para ejecutarse.

Idea clave: el Scheduler organiza la ejecución de las tareas para que las de mayor prioridad reciban atención preferente, sin depender de que las tareas de menor prioridad cedan voluntariamente el procesador.



Prioridades

En FreeRTOS, las prioridades indican qué tarea tiene preferencia para utilizar el procesador. Se representan mediante valores numéricos: cuanto mayor es el número, mayor es la prioridad.

Por ejemplo, una tarea de prioridad 5 para atender una emergencia tendrá preferencia sobre una tarea de prioridad 1 que hace parpadear un LED, siempre que ambas estén listas para ejecutarse.

La prioridad no hace que una tarea se ejecute más rápido; determina cuál recibe atención primero cuando varias tareas están listas. Una tarea de mayor prioridad que esté bloqueada no impide que se ejecuten otras tareas listas.



Multitarea y el Planificador (Scheduler)

La gracia de un RTOS está en su capacidad para simular multitarea.

Aunque tengamos un solo núcleo en el procesador, el RTOS nos permite dividir nuestro código en pequeñas unidades independientes llamadas Tareas (Tasks).

El componente encargado de gestionar esto es el Scheduler (Planificador). El Scheduler detiene una tarea, guarda su estado, carga el estado de la siguiente tarea y la ejecuta.

Esto ocurre tan rápido (cientos o miles de veces por segundo) que nos da la ilusión de que todo se ejecuta simultáneamente.



Multitarea Apropiativa (Preemptive)

FreeRTOS utiliza normalmente un modelo de multitarea apropiativa. Esto significa que el Scheduler puede interrumpir (apropiarse de) la tarea que se está ejecutando en cualquier momento para dar paso a otra tarea de mayor prioridad.

Imagina este escenario:

- Tenemos una tarea **TareaPantalla** dibujando gráficos (prioridad baja).
- Llega una interrupción de un sensor crítico y desbloquea **TareaSensor** mediante una API segura para ISR.
- El Scheduler **pausa inmediatamente** la **TareaPantalla**.
- Ejecuta la **TareaSensor** (prioridad alta).
- Al terminar, reanuda **TareaPantalla** exactamente donde se quedó.

Esto es algo que con un “Super Loop” es extremadamente difícil de conseguir de forma limpia.

Diferencias: Super Loop vs RTOS

Para visualizarlo mejor, veamos una comparativa directa:

CARACTERÍSTICA	SUPER LOOP (BARE METAL)	RTOS (FREERTOS)
Estructura	Un bucle `while(1)` gigante	Múltiples `Tasks` independientes
Gestión de tiempo	Esperas que detienen el flujo o lógica manual con `millis()`	Esperas que bloquean solo la tarea actual
Complejidad	La coordinación se enreda al crecer	Las responsabilidæades se separan en tareas
Respuesta	Depende de la duración del bucle	Depende de prioridades, bloqueos e interrupciones
Uso de CPU	Puede desperdiciar CPU con espera activa	La CPU puede entrar en `Idle` si no hay trabajo